UNIVERSITY OF DEBRECEN
FACULTY OF INFORMATICS

THESIS

Development of a Web-Based School Management and Electronic Diary System

Supervisor:
Dr. Vágner Anikó Szilvia

Candidate:
Mikhail Zviagintsev
Computer Science

DEBRECEN
2026

Table of Contents

1 Introduction **3**

1.1 Motivation and context 3

1.2 Problem statement 3

1.3 Aim and objectives 4

1.4 Method and scope 5

2 Domain model and methods **6**

2.1 The five-point scale 6

2.2 Subject averages and final marks 6

2.3 Attendance percentage 7

2.4 Converting legacy scores 7

2.5 Related systems and techniques 8

3 Requirements and problem analysis **9**

3.1 Actors and intended workflows 9

3.2 Functional requirements 9

3.3 Non-functional requirements 10

3.4 Data quality and acceptance criteria 11

3.5 Scope decisions 11

4 System design and architecture **13**

4.1 Application structure 13

4.2 Responsibility boundaries 14

4.3 Relational data model 15

4.4 Integrity and query design 16

4.5 Security design 17

4.6 Persistence, scheduling and deployment 17

5 Implementation **19**

5.1 Assembly and dependency structure 19

5.2 Five-point marks 19

5.3 Attendance and timetable rules 21
5.4 School news and the home page 21
5.5 The diary interface 23
5.6 Reports, export and demonstration data 25

6 Testing and performance evaluation 26
6.1 Test strategy and experimental setup 26
6.2 Automated test results 26
6.3 Verification of the five-point rule 27
6.4 Migrations and backup 27
6.5 Load test 28
6.6 Browser and interface checks 29
6.7 Security checks 29
6.8 Defects found during final verification 30
6.9 Assessment and limitations 30

7 Conclusion and future work 32
7.1 Application outcomes 32
7.2 Testing results 32
7.3 Possible extensions 33
7.4 Final conclusion 33

Bibliography 34

Declaration on the Use of Artificial Intelligence Tools 35

1 Introduction

1.1 Motivation and context

A school produces a steady stream of small records: a mark given in a lesson, a pupil who was late, a room that changes for one day, a reminder about a parent meeting. Each record is trivial. Together they decide whether a pupil, a parent and a teacher share the same picture of how a term is going. On paper this picture lives in a class register, a pupil's diary and a notice board, and it is only as current as the last time somebody copied information from one to another.

Electronic diaries address this problem by keeping one authoritative record and showing it in the form that each reader needs. In several countries they are everyday tools. Russian services such as dnevnik.ru popularised a simple layout: the pupil opens a week, sees each day's lessons in order, and sees the marks received in those lessons next to them. Hungarian schools, like Russian ones, grade on a scale from one to five, so a system built around that scale suits a school in either setting.

My motivation was to build such a system as a complete piece of software engineering rather than as a collection of screens. A diary looks simple, but it combines persistent storage, access rules that depend on who is asking, arithmetic that must be explainable (what exactly is a subject mark?), scheduled work such as reminders, and an interface that parents will open on a phone. These concerns fit naturally into one project in which every design decision can be tested.

The result is a web application for a single school. Administrators manage accounts, classes, subjects, the timetable and school news. Teachers record attendance and marks. Pupils and parents read a weekly diary, their marks and attendance. Visitors who are not signed in see a public home page with the latest school news.

1.2 Problem statement

The central difficulty of a school system is not storing data but deciding who may see and change which data, and presenting the result in a way that its readers trust. Three problems follow from this.

First, *access depends on relationships*, not only on roles. A teacher may record a mark for a pupil only if the pupil belongs to a class that the teacher teaches. A parent may read the records of their own children and nobody else's. A pupil may read only their own

records. These rules are shared by several features, so scattering them across request handlers would invite inconsistencies.

Second, *the grading scale must be unambiguous*. The first version of the system stored marks as scores from 0 to 100 and derived a weighted grade-point average. That scheme did not match the way the intended users talk about marks. A pupil or parent expects a whole mark from 1 to 5, a subject mark that is an average rounded in a stated way, and a colour-coded presentation that can be read at a glance. Changing the scale also affects existing data, so it needs a migration with defined behaviour.

Third, *the first screen must be useful to everyone*. A school's most common communication is an announcement, and an administrator should be able to publish one without help. Placing a public news feed at the root of the site also changes the routing of the application: the role-specific start page has to move elsewhere, and the news text, which is written by a person, has to be displayed without ever being interpreted as markup.

1.3 Aim and objectives

The aim is to develop and evaluate a web-based school management system with a five-point marking scheme, a weekly electronic diary and a public school-news home page.

The objectives are:

- to design a relational data model with explicit integrity rules for accounts, classes, subjects, timetable, attendance, marks, notifications and news;
- to centralise the relationship-based access rules in one component that is testable without an HTTP layer;
- to replace the original 0–100 scoring with whole marks from 1 to 5, including validation, aggregation rules and a defined migration of existing data;
- to build a server-rendered interface in the style of a school diary, with a weekly view that shows marks next to lessons and a layout that works on a phone;
- to provide a public home page with news that administrators can publish, pin, edit and delete;
- to evaluate the result with automated unit, integration and browser tests, a load test against the target performance requirement, and checks of the security mechanisms.

The evaluation asks whether the implemented rules hold under test, whether the system meets its response-time requirement on a school-sized data set, and what the evidence does not establish. Chapters 5 and 6 explain the implementation and the results.

1.4 Method and scope

The project follows an implementation-based method. Requirements (Chapter 3) determine the architecture (Chapter 4); the implementation (Chapter 5) is verified by tests at several levels (Chapter 6). The tests run against two interchangeable storage back ends: in-memory repositories for fast feedback, and PostgreSQL for the real system. The same HTTP scenarios are executed against both, so the tests exercise the production routes, middleware and handlers.

The recorded results were produced on 19 October 2026. The application source and the documentation are available in the project repository [1].

The system is designed for one school. It is a web application served by a single process and a PostgreSQL database. Fee and payment tracking, native mobile applications, integration with external identity providers and any user study with real pupils and teachers are outside the scope of the delivered work. Where a limitation affects a result, it is stated at that point and collected in Section 6.9.

2 Domain model and methods

2.1 The five-point scale

The system uses the five-point scale in the form familiar in Russian and Hungarian schools. A mark is a whole number: 5 is excellent, 4 good, 3 satisfactory, 2 poor and 1 very poor. Table 2.1 lists the verbal meaning and the colour used in the interface. The colours form a sequence from green to dark red so that a row of marks can be read as a pattern without reading the digits.

Table 2.1: The five-point scale and its presentation

Mark	Meaning	Colour in the interface
5	excellent	green
4	good	light green
3	satisfactory	amber
2	poor	red
1	very poor	dark red

Two design decisions follow from using whole marks. A mark of 0, a mark above 5 and a fractional mark are all invalid input; they are rejected rather than rounded, because silently correcting a teacher's entry would change a decision the teacher made. Averages, by contrast, are fractional and are shown with two decimals; the whole-number *final* mark is derived from them by an explicit rule (Section 2.2).

2.2 Subject averages and final marks

Let $m_1, \dots, m_n$ be the marks a pupil received in one subject, optionally restricted to one term. The subject average is the arithmetic mean

$$\bar{m} = \frac{1}{n} \sum_{i=1}^{n} m_i. \tag{2.1}$$

The final subject mark rounds the average to the nearest whole mark, with halves rounding up, and keeps it inside the scale:

$$F(\bar{m}) = \min\bigl(5,\ \max\bigl(1,\ \lfloor \bar{m} + \tfrac{1}{2} \rfloor\bigr)\bigr), \qquad F = 0 \text{ when } n = 0. \tag{2.2}$$

Thus an average of 4.5 gives a final mark of 5, while 4.49 gives 4. A subject with no marks has no final mark; the value 0 stands for “none” and never appears as a mark that can be entered.

The overall average of a pupil is the mean of *all* marks, not the mean of the subject averages. A subject with many marks therefore weighs more than one with few. This is the simpler and more common reading of “average mark” for a pupil, and the interface labels it as such. The earlier credit-weighted grade-point average was removed together with the 0–100 scores; no field of the API refers to it any more.

2.3 Attendance percentage

Each pupil has one attendance record per class and day with one of four statuses: present, absent, late or excused. Let P, L, E and T be the numbers of present, late and excused days and of all recorded days in the period. The attendance percentage counts lateness as attendance and removes excused days from the base:

$$A = \frac{P + L}{T - E} \times 100. \tag{2.3}$$

When $T - E \le 0$ there is nothing to penalise and $A = 100$. The percentage is computed over a period (the last 30 days in the diary) and rounded to two decimals.

A low-attendance warning is generated when A falls below 75 % and at least five days have been recorded. The warning fires once, at the moment the percentage *crosses* the threshold, so that recording one more absence for a pupil who is already below the threshold does not send another message.

2.4 Converting legacy scores

Databases created before the change contain scores from 0 to 100. The migration maps each score to a mark with the bands of Table 2.2, which are the conventional ones for a percentage-to-five-point conversion.

Table 2.2: Conversion of legacy 0–100 scores applied by migration 000003

Legacy score	New mark
90–100	5
75–89	4
60–74	3
40–59	2
below 40	1

The conversion loses information: two scores in one band become the same mark. The reverse migration therefore maps each mark back to the middle of its band (5 to 95, 4 to 82, 3 to 67, 2 to 50, 1 to 20) and is documented as lossy. A reversible conversion does not exist, and pretending otherwise would hide the loss.

2.5 Related systems and techniques

Electronic diaries are well established. Services such as dnevnik.ru present a weekly view in which lessons and marks appear together, and they combine it with news and messages. This project borrows that interaction model, namely the week as the main unit and the mark placed next to its lesson. The visual design, text and structure are original, and no branding or content of any existing service is reproduced.

The implementation relies on a small number of well-documented techniques. Passwords are hashed with bcrypt [5]. Sessions are signed JSON Web Tokens [4]. State-changing browser requests are checked against the request origin as defence in depth against cross-site request forgery [6]. Exported tables neutralise cells that a spreadsheet could interpret as formulas [7]. Database constraints enforce invariants that application code could otherwise bypass [2]. The browser interface uses HTML fragments returned by the server and swapped in by htmx [9], which keeps the application free of a separate front-end build and lets it work with JavaScript disabled for ordinary navigation and forms.

3 Requirements and problem analysis

3.1 Actors and intended workflows

The system has five kinds of actor. Table 3.1 summarises what each may do. The rules that decide “own” and “linked” are relationships stored in the database, not properties of the account itself, which is why the access component of Section 4.2 needs repository access.

Table 3.1: Actors and their permissions

Actor	Permitted activity
Guest	Read the public home page and individual news items; sign in.
Student	Read own weekly diary, marks, attendance, timetable and notifications; export own reports.
Parent	The same as a student, for each linked child.
Teacher	Record attendance and marks for classes they teach (as homeroom teacher or through a timetable lesson); change or delete only marks they recorded; read class reports and statistics.
Administrator	Manage users, classes, subjects, teachers’ qualifications, parents’ links, timetable, news and announcements; full read access to all reports.

A typical day follows the roles. In the morning a teacher opens the attendance sheet of the class, marks absences and saves. During the day the teacher presses a digit in the gradebook to give a mark; the pupil and the parents receive a notification. In the evening the pupil opens the diary for the week and sees the mark beside the lesson. At any time an administrator can publish an announcement that appears on the home page and, if sent as an announcement, in the notifications of the chosen roles.

3.2 Functional requirements

Table 3.2 lists the functional requirements together with the evidence that verifies each of them. Identifiers are used in the test names and in the documentation of the repository.

Table 3.2: Functional requirements and verification evidence

ID	Requirement	Evidence
FR-1	Sign in with e-mail and password; self-registration is closed and accounts are created by an administrator.	Authentication scenario
FR-2	Roles are enforced on every endpoint and page; data access is limited to the user's own scope.	Role-based access scenario; access-control unit tests
FR-3	Administrators search, edit, deactivate and delete users and reset passwords.	User service tests
FR-4	Create, read, update and delete classes, subjects, teachers (with their subjects), students and parents with links to children.	API scenarios
FR-5	Timetable per class and teacher; overlapping lessons of a class, a teacher or a room, and subjects the teacher does not teach, are rejected.	Timetable service tests
FR-6	Attendance is recorded per class and day with four statuses; reports show the percentage.	Attendance tests
FR-7	Marks are whole numbers from 1 to 5; the subject mark is the average rounded half up; anything else is rejected.	Grade service tests; five-point scenario
FR-8	Pupils and parents read a weekly diary with marks next to lessons; teachers use a gradebook per class and subject.	Web tests; browser test
FR-9	Notifications for new marks, absences and announcements, optionally also by e-mail; a daily reminder for teachers who have not recorded today's attendance.	Notification and reminder tests
FR-10	Export marks and attendance to CSV that opens correctly in a spreadsheet.	Export tests
FR-11	A public home page lists school news; administrators publish, pin, edit and delete items.	News tests; browser test
FR-12	Aggregated marks and attendance are available as database views for reporting.	Migration 000005

3.3 Non-functional requirements

Security is the first concern because the data describe minors. Passwords must be stored only as bcrypt hashes. Session cookies must be `HttpOnly` and `SameSite=Strict`. Every state-changing browser request must prove that it originates from the application. Output must be escaped, a content-security policy must forbid foreign scripts, and repeated login attempts must be limited.

Data integrity must not depend on the application alone. Marks outside the scale, lessons that end before they start and duplicate natural keys must be impossible at the database level as well.

Performance. Read endpoints should answer in under 200 ms at the 95th percentile with 20 concurrent users on a data set of about 1 000 pupils (NFR-3). This is the requirement evaluated in Section 6.5.

Usability. A user should be able to complete the main task of their role without help, and the layout must work from 360 px width. The first part is covered by a usability test plan (Section 6.9), the second by automated and manual checks.

Maintainability and recoverability. The code must keep handlers, business rules and persistence in separate layers, with services depending on interfaces. A tested backup and restore procedure must exist, with a recovery-point objective of 24 hours. The system must run from a container image together with PostgreSQL 16.

3.4 Data quality and acceptance criteria

The business rules above are enforced in two places. The service layer validates input and returns an explicit error that handlers translate into an HTTP status: 400 for a malformed, out-of-range or conflicting value (an overlapping lesson is reported this way, with a message naming the clash), 403 for a forbidden action and 404 for an unknown object. The database rejects what slips past: the `chk_grades_five_point` constraint accepts only the values 1 to 5, a check constraint requires a lesson to end after it starts, and unique constraints guard e-mail addresses, subject codes and student numbers. The service rule gives a helpful message, the constraint is the last line of defence.

The work is accepted if the application runs from a clean database with the supplied migrations; every functional requirement of Table 3.2 has a passing automated check or a recorded manual check; the migrations apply forwards, backwards and forwards again without error; and the performance requirement is met on a documented data set. Acceptance does not require a user study or a statement about capacity beyond the measured configuration; Section 6.9 states what remains unproven.

3.5 Scope decisions

Public self-registration was deliberately removed. In the first version anyone could register and obtain the administrator role, which is unacceptable for a system holding pupils' data. Accounts are now created by an administrator, and the first administrator is bootstrapped from two environment variables at start-up.

Fee tracking is optional in the original work plan and is not delivered. A real-time

messaging feature was also left out: notifications cover the main need, and a chat between teachers and parents would raise moderation questions that a thesis-sized project cannot address responsibly. The user interface is written in English; making it multilingual is listed as future work.

4 System design and architecture

4.1 Application structure

Figure 4.1 shows the structure. A browser talks to one Go process over HTTP. The process serves two interfaces from the same services: a JSON API under `/api` and server-rendered HTML pages. PostgreSQL stores all state. The optional mail server is the only other external dependency, and e-mail can be switched to a log-only mode for demonstration.

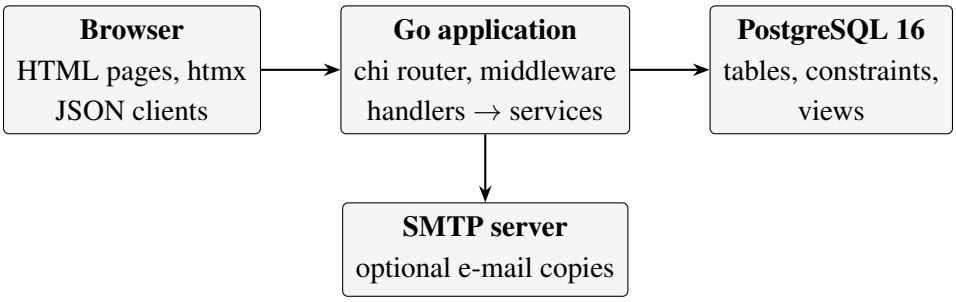


Figure 4.1: Application structure and responsibility boundaries

The code follows a layered architecture, shown in Figure 4.2. Each request passes through middleware (security headers, request logging, rate limiting for login, authentication and role checks), a handler that parses the request and formats the response, a service that contains the business rules, and a repository that contains the SQL. Dependencies point downwards only. Services depend on repository *interfaces* declared next to them, so the same services run on PostgreSQL repositories in production and on in-memory repositories in tests.

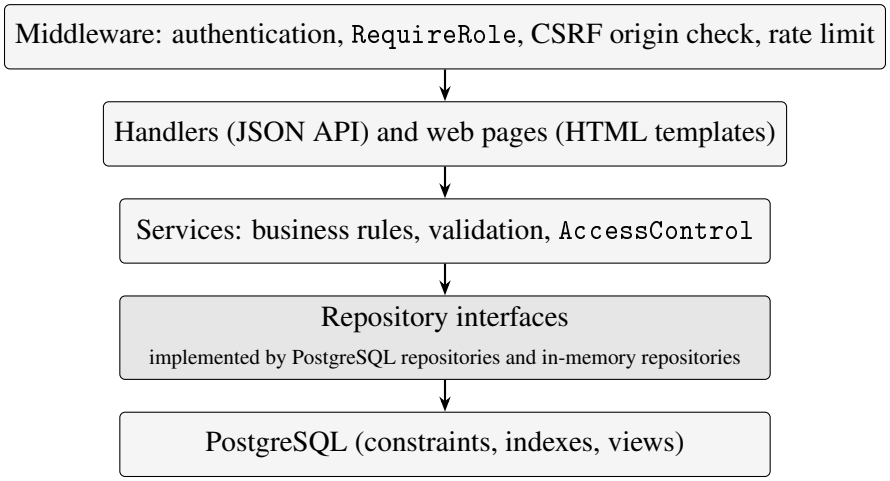


Figure 4.2: Layers of the application; dependencies point downwards

The main source directories are `internal/handlers` and `internal/web` (presentation), `internal/services` (rules), `internal/repository` and `internal/memrepo` (persistence), `internal/models` (shared types) and `internal/app`, which assembles the router so that the production binary and the tests use exactly the same routes. The commands `cmd/api`, `cmd/seed` and `cmd/loadtest` provide the server, a demonstration-data loader and a load generator.

4.2 Responsibility boundaries

Handlers contain no business rules. They translate a request into a service call and a service error into a status code through one helper: a forbidden action becomes 403, a missing object 404 and everything else that is a validation error 400. Services receive an explicit `Actor` value holding the user identifier and role instead of reading the HTTP request. This makes the access rules testable by constructing an actor in a unit test.

Centralised access control

The component `AccessControl` answers two questions used by attendance, grades and the timetable: may this actor record data for this class, and may this actor read the records of this pupil? An administrator may do both. A teacher may record for a class if they are its homeroom teacher or have at least one timetable lesson with it, and may read the pupils of such classes. A pupil may read only their own records. A parent may read the records of pupils linked to them. Everything else is denied. Because every service calls the same two functions, adding a feature cannot silently introduce a different interpretation of “the teacher of a class”.

Server-rendered interface

The interface is generated by Go `html/template` and embedded into the binary with `go:embed`, so the container needs no separate static-file server and the build has no JavaScript toolchain. The htmx library is vendored and served from the application itself, which allows a strict content-security policy that permits scripts only from the same origin. Forms are ordinary HTML forms that work without scripting; htmx only replaces the page body to avoid a full reload.

4.3 Relational data model

The schema has fourteen tables, three reporting views and five migrations. Figure 4.3 shows the principal entities and Table 4.1 their purpose and constraints.

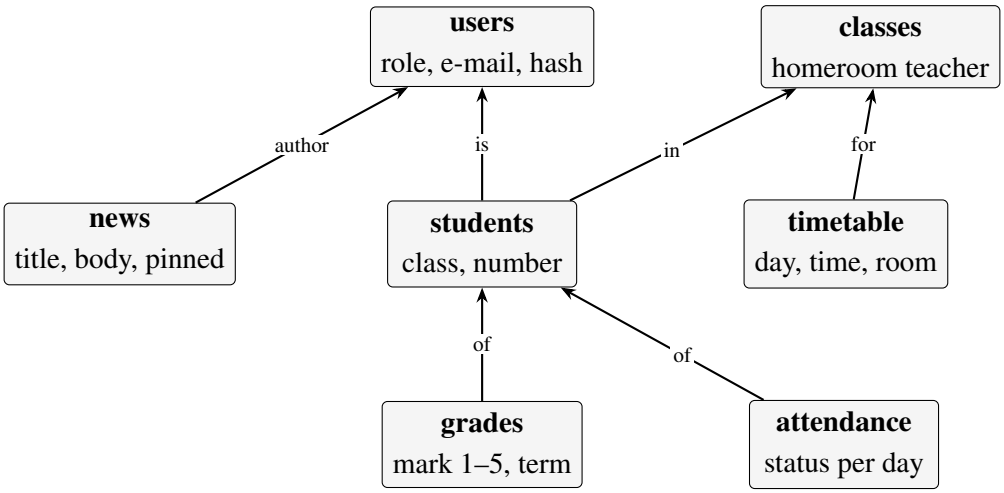


Figure 4.3: Principal entities (teachers, parents, subjects, link tables and notifications omitted)

Table 4.1: Principal tables and their constraints

Table	Purpose	Important constraint
users	Account, role, active flag, soft-delete timestamp	Unique e-mail; role is an enumerated type
teachers, students, parents	Role-specific data referring to a user	Unique `user_id`; foreign key to users
classes	Class name, level, section, homeroom teacher	Foreign key to users
subjects	Subject name and code	Unique code
teacher_subjects, student_parents	Qualifications and parent–child links	Composite keys
timetable	Weekly lesson of a class	Check: end after start; day 0–6
attendance	One status per pupil, class and day	Unique (pupil, class, date)
grades	A mark with subject, class, term and author	`chk_grades_five_point`: mark in $\{1,\ldots,5\}$
notifications	Message to a user with read flag	Index on unread messages
news	School news item	Index for the feed (pinned, then newest)

The schema is in third normal form. Account data (e-mail, password hash, name, role) is stored once in `users`; the role tables keep only role-specific columns. Names of classes, subjects and teachers are never copied into marks, attendance or timetable rows, they are reached through foreign keys. Averages and percentages are not stored but computed, so they cannot become stale. One deliberate denormalisation exists: `grades` and `attendance` record the `class_id` the pupil belonged to *at the time*, because a pupil may later change class and the historical fact must not move with them. Users are soft-deleted so that the marks of a removed account are preserved.

4.4 Integrity and query design

Primary keys, foreign keys, unique and check constraints enforce different aspects of consistency [2]. The five-point rule is an example of defence in depth: the service rejects an invalid mark with a message that a teacher can act on, and the check constraint ensures that no other code path, script or manual SQL statement can store one.

Indexes follow the dominant queries: marks by pupil and term for the diary and

reports, timetable by teacher and day for conflict detection, notifications by user and read state for the bell counter, attendance by pupil and date, and the news feed ordered by pinned flag and creation time. Three views (`v_student_grade_summary`, `v_student_attendance_summary`, `v_class_overview`) provide aggregated marks and attendance for reporting tools without duplicating the logic of Chapter 2 outside the application; the attendance view uses the same formula as Equation (3).

4.5 Security design

Authentication. Passwords are hashed with bcrypt at its default cost. A successful login returns a signed JSON Web Token valid for 24 hours; the web interface stores it in an `HttpOnly`, `SameSite=Strict` cookie (with the `Secure` flag behind HTTPS). On every web request the user is loaded again, so a deactivated or deleted user is signed out immediately even if the token has not expired.

Cross-site request forgery. Every state-changing web request (anything other than GET and HEAD) must carry an `Origin` or `Referer` header whose host equals the request host; otherwise it is rejected with 403. This complements the `SameSite` cookie attribute [6].

Injection and scripting. SQL statements are parameterised. Templates escape all dynamic text, and the news body is stored and displayed as plain text split into escaped paragraphs, so an administrator cannot accidentally publish markup. Responses carry a content-security policy (`default-src 'self'`, `script-src 'self'`, `frame-ancestors 'none'`), `X-Frame-Options: DENY`, `X-Content-Type-Options: nosniff` and a same-origin referrer policy [8].

Brute force. Login requests are limited per client address with a fixed window (10 per minute by default, configurable); excess requests receive 429 with a `Retry-After` header. Public self-registration does not exist.

4.6 Persistence, scheduling and deployment

Notifications and reminders. Recording a mark or an absence creates in-app notifications for the pupil and the linked parents. An optional mailer sends a copy by SMTP or, in demonstration mode, writes it to the log. A reminder job runs at a configurable hour (15:00 by default) and notifies teachers who have not recorded attendance for the day.

Migrations. Schema changes are plain SQL files in `migrations/`, each with an `up` and a `down` script in the format of the migration tool [10]. The continuous-integration pipeline applies all scripts forwards, backwards and forwards again against a clean database.

Container. A multi-stage Docker build compiles the server and the seeding tool with

the Go toolchain and copies the binaries into a small Alpine image that runs as a non-root user and includes time-zone data, so that the reminder job follows local school time. Docker Compose starts the application together with PostgreSQL 16.

Backup and restore. The script `scripts/backup.sh` writes a compressed custom-format dump with `pg_dump` [3] to a temporary name, renames it when complete, verifies it with `pg_restore -list` and removes dumps older than a configurable number of days. The script `scripts/restore.sh` restores a dump into an empty database and stops at the first error. The objectives are a recovery point of 24 hours (daily dump) and a recovery time of under 30 minutes for a school-sized database; Section 6.4 reports a restore test.

5 Implementation

5.1 Assembly and dependency structure

The function `app.NewRouter` receives a set of ready-made services and returns the complete HTTP handler: public routes, the JSON API under `/api` and the web interface. The production entry point `cmd/api` builds the services on PostgreSQL repositories; the tests build the same services on in-memory repositories or on a real database and call the same function. As a result no test uses a hand-written approximation of the router, and a route that is missing in production is missing in the tests as well.

The server reads its configuration from environment variables (Table 5.1). The first administrator is created at start-up from `ADMIN_EMAIL` and `ADMIN_PASSWORD` if no such account exists, which replaces the removed public registration.

Table 5.1: Main configuration variables

Variable	Meaning
`DATABASE_URL`	PostgreSQL connection string
`JWT_SECRET`	Signing key for sessions; must be changed in production
`PORT`	HTTP port (8080 by default)
`ADMIN_EMAIL`, `ADMIN_PASSWORD`	Bootstrap administrator
`COOKIE_SECURE`	`true` when served over HTTPS
`LOGIN_RATE_LIMIT`	Login attempts per minute per address (10 by default)
`REMINDER_HOUR`	Hour of the attendance reminder; -1 disables it
`SMTP_*`, `EMAIL_LOG`	E-mail delivery, or log-only mode for demonstrations

5.2 Five-point marks

The change from 0–100 scores to marks from 1 to 5 touches four layers, and each layer has its own check.

Service. Validation is a single function that rejects anything that is not a whole number in range, including NaN. The final mark of a subject is computed by one function used by every report (Listing 5.1). The request type keeps the mark as a floating-point number so that a client sending `3.5` receives a clear message instead of a decoding error.

Listing 5.1: Validation and final mark (simplified from `grade_service.go`)

```
const (MinMark = 1; MaxMark = 5)

func validateMarks(m float64) error {
    if math.IsNaN(m) || m != math.Trunc(m) || m < MinMark || m > MaxMark {
        return fmt.Errorf("mark must be a whole number from %d to %d", MinMark,
     MaxMark)
    }
    return nil
}

func FinalMark(avg float64) int {
    if avg <= 0 { return 0 }                // no marks
    f := int(math.Floor(avg + 0.5))         // half rounds up: 4.5 -> 5
    return min(MaxMark, max(MinMark, f))
}
```

Database. Migration 000003 first converts existing scores with the bands of Table 2.2, then replaces the old range constraint by `chk_grades_five_point`, which accepts only 1, 2, 3, 4 and 5. The order matters: adding the constraint first would fail on legacy rows. The down migration removes the new constraint, maps each mark to the middle of its band and restores the old one.

Reports. A report contains the individual marks, one row per subject with the average, the final mark and the number of marks, and the overall average (Section 2.2). The earlier grade-point and credit-weighted fields were removed from the API, so a client that still reads them fails visibly instead of showing stale semantics.

Permissions. Any administrator or teacher with access to the class may give a mark; only an administrator or the teacher who recorded a mark may change or delete it. Recording a mark creates a notification for the pupil and the linked parents.

Interface. In the gradebook (Figure 5.1) the teacher chooses a class, a subject and a term and sees one row per pupil with the marks in date order, the average and five coloured buttons labelled 5 to 1. Pressing a button gives that mark immediately; clicking an existing mark opens it for change or removal. The five-point scale is repeated as a legend on the pupil's marks page and on the home page.

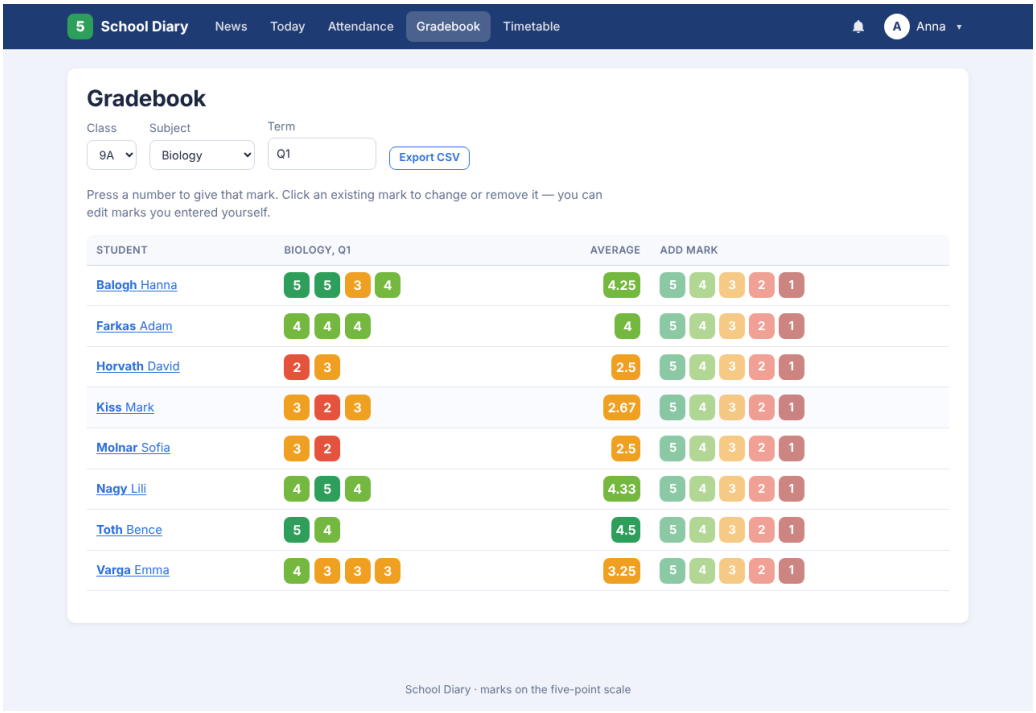


Figure 5.1: Teacher's gradebook: marks by pupil with averages and one-press mark entry

5.3 Attendance and timetable rules

Attendance is recorded in bulk for a class and a day; recording again for the same day overwrites the previous status, which matches the way a teacher corrects a sheet. The percentage follows Equation (3). The low-attendance check compares the percentage before and after the record and fires only when it crosses the 75 % threshold with at least five recorded days.

The timetable rejects three kinds of conflict: two lessons of the same class, of the same teacher or in the same room whose time intervals overlap on the same weekday. Intervals are treated as half-open, so a lesson ending at 09:40 and another starting at 09:40 do not conflict. A lesson is also rejected if the teacher has not been qualified to teach the subject. The conflict test is a pure function over the lessons of one day, which makes it easy to test exhaustively.

5.4 School news and the home page

The new home page required a change in routing. The root path `/` became public and shows the news feed; the role-specific start page "Today" moved to `/dashboard`, and a successful login redirects there. A signed-in user sees the same home page inside the normal menu, a guest sees a sign-in button and a short explanation (Figure 5.2).

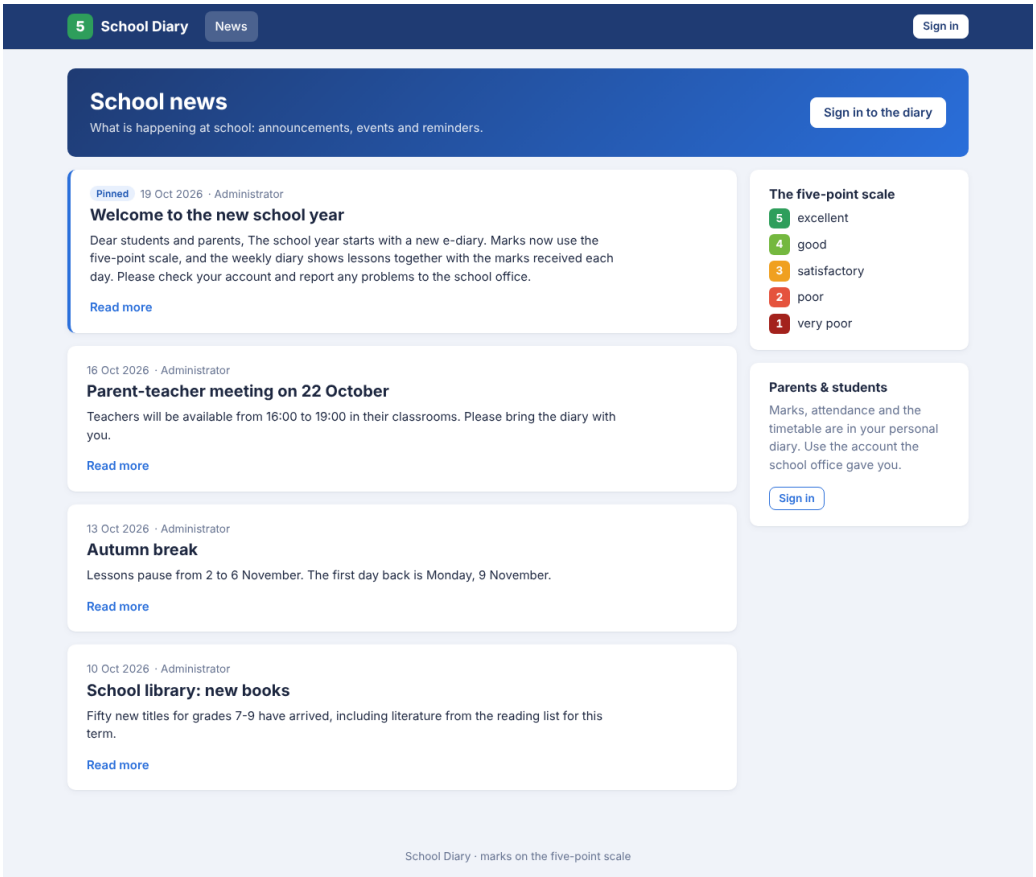


Figure 5.2: Public home page with the school news feed and the five-point legend

News items live in the table `news` with a title (up to 200 characters), a text (up to 10 000 characters), a pinned flag, an author and timestamps. The feed is ordered by pinned flag and then newest first, which is exactly what the index `idx_news_feed` supports. The JSON API exposes `GET /api/news` publicly and `POST`, `PUT` and `DELETE` to administrators only; the service rejects anyone else with 403 even if a handler were misconfigured.

The text is plain text. The page splits it into paragraphs at blank lines and escapes each paragraph, so markup typed by an administrator is shown literally. A feed card shows the beginning of the text collapsed into one line and shortened to about 260 characters at a word boundary. An earlier version showed only the first paragraph, which made a news item that begins with a greeting look empty; the final version joins the paragraphs, and a unit test covers this case. Administrators manage news on a separate page (Figure 5.3) with a form for a new item and a table of published items with edit and delete actions.

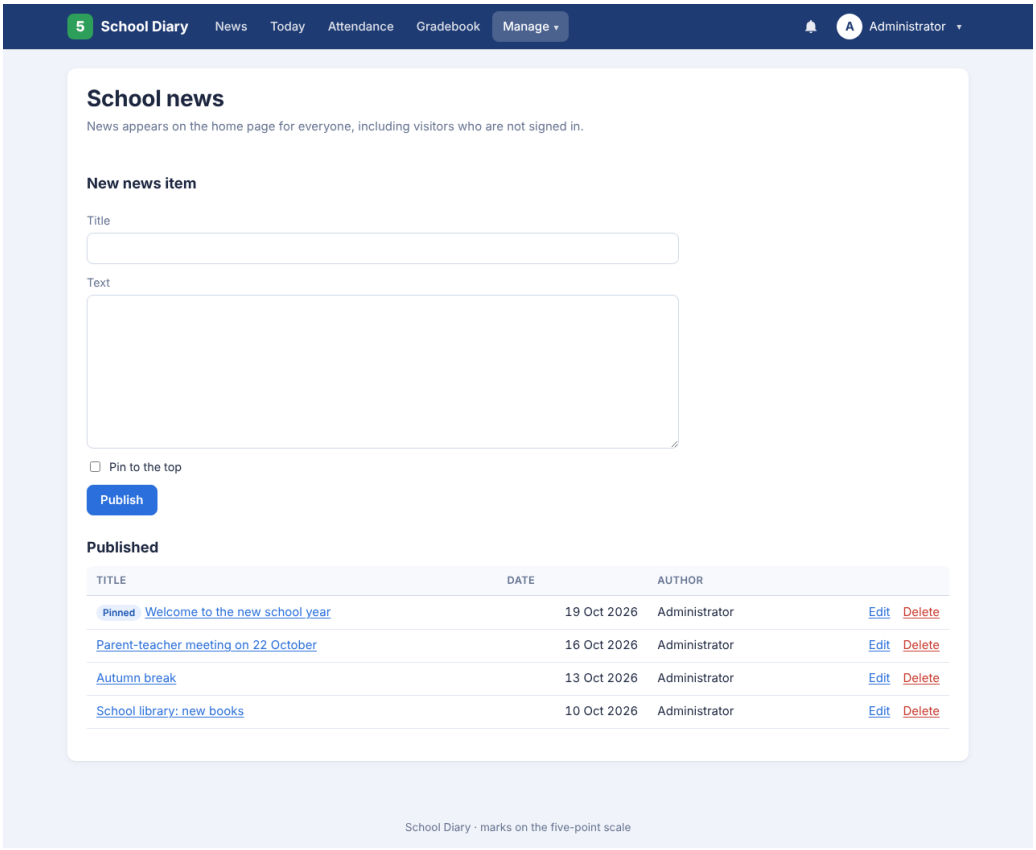


Figure 5.3: Administrator's news page: publish form and list of published items

5.5 The diary interface

The interface replaces the earlier side-bar layout with a top bar and white cards on a light background. The menu depends on the role: pupils see News, Today, Diary, Marks, Attendance and Timetable; teachers see Attendance and Gradebook; administrators additionally have a Manage menu. A bell with an unread counter links to the notifications.

The central page is the weekly diary (Figure 5.4). The function that builds it takes the Monday of the requested week, the pupil's class timetable and the pupil's marks and returns one block per weekday. Saturday and Sunday appear only when they contain lessons or marks. A mark is placed beside the *first lesson of its subject on the day the mark was given*; marks for which no lesson exists that day are listed under "Other marks" in the same card, so no mark is ever hidden. Today is highlighted, and the buttons "Previous", "This week" and "Next" move through weeks through a query parameter, which also makes a particular week bookmarkable. Parents reach the same view for each child.

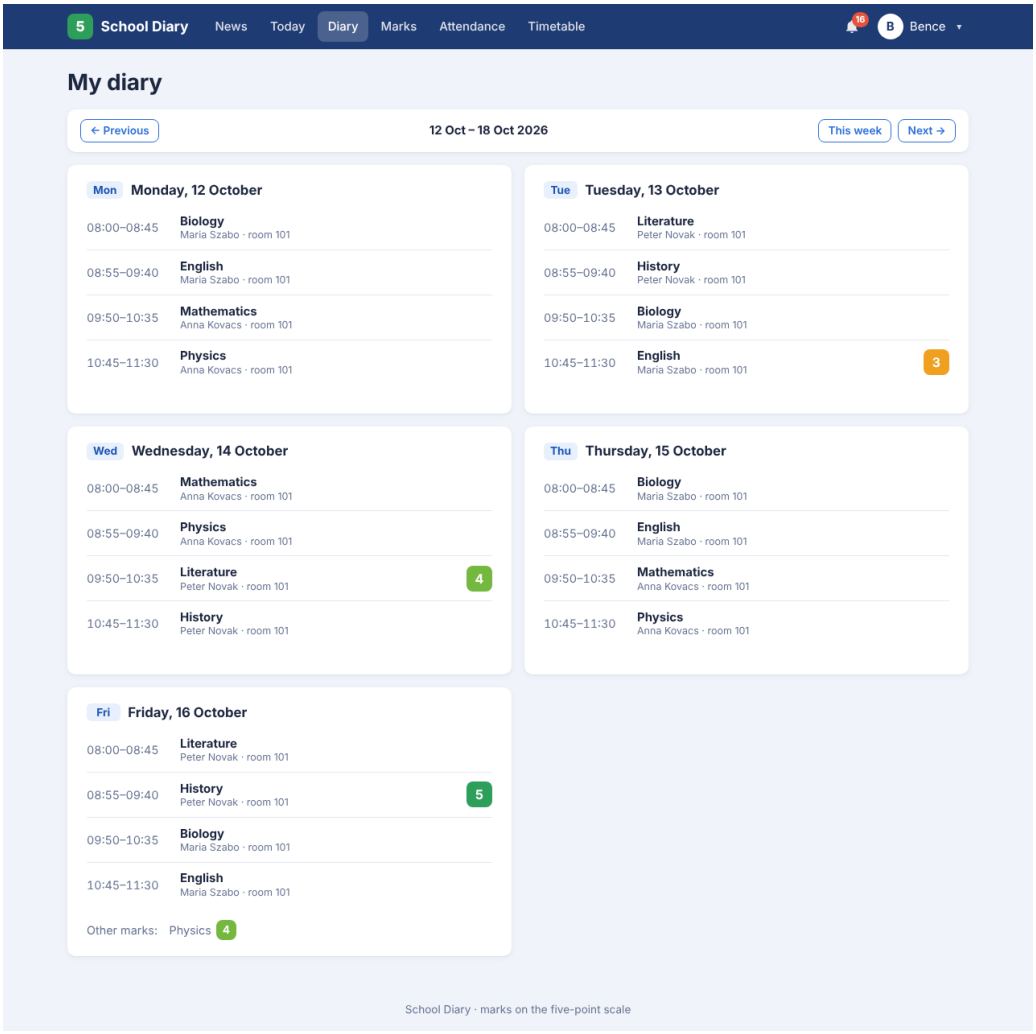


Figure 5.4: Weekly diary of a pupil: lessons by day with the marks received

The marks page (Figure 5.5) shows the overall average, the number of marks and subjects, and for each subject all marks in date order, the average and the final mark. A caption explains that the final mark is the average rounded half up.

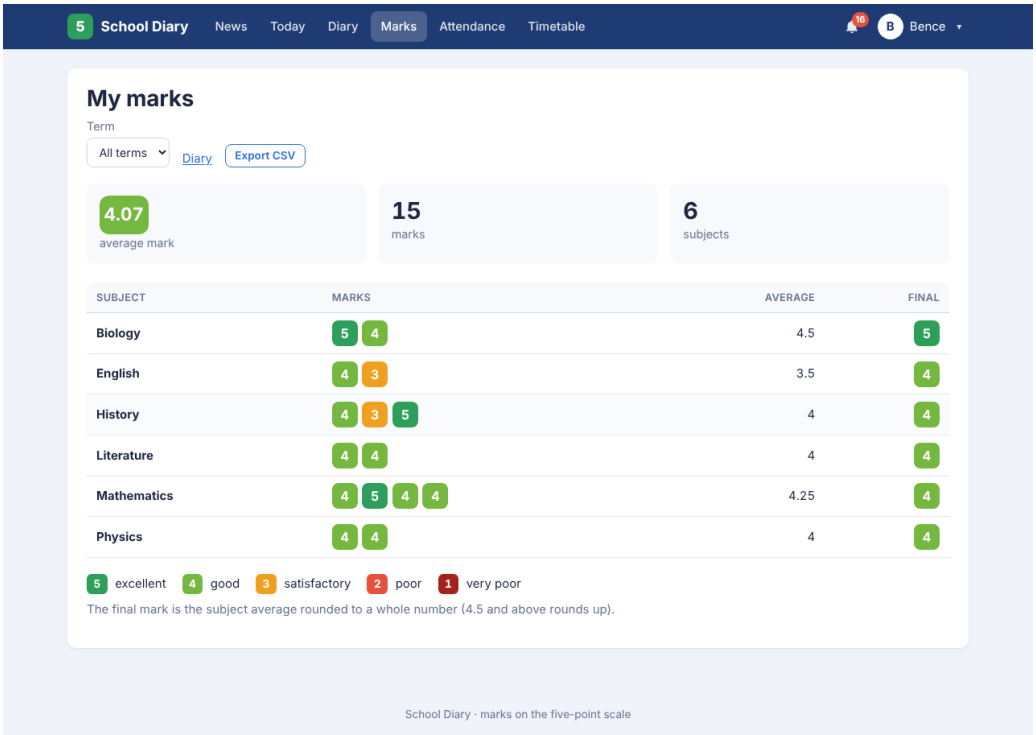


Figure 5.5: A pupil's marks by subject with averages and final marks

Responsive behaviour is achieved with CSS grid and wrapping flex rows. On a viewport 390 px wide the home page does not scroll horizontally (Section 6.6). Interface text is English; all strings are in templates, which keeps a later translation mechanical.

5.6 Reports, export and demonstration data

Marks and attendance can be exported as CSV for a pupil and for a whole class. The writer starts the file with a UTF-8 byte-order mark so that spreadsheet programs detect the encoding, and prefixes any cell that begins with `=`, `+`, `-`, `@`, a tab or a carriage return with an apostrophe so that it cannot be executed as a formula [7]. The export service reuses the report functions, which keeps the exported averages identical to the screen.

The command `cmd/seed` loads a demonstration school into an empty database: an administrator, teachers, two classes (9A and 9B) with 16 pupils in total, parents, subjects, a weekly timetable, marks from 2 to 5 spread over the last school days, attendance and four news items. It exercises every repository against real SQL and gives every role an account to try.

6 Testing and performance evaluation

6.1 Test strategy and experimental setup

Testing has five levels (Table 6.1). The unit tests run without any infrastructure and finish in seconds. The HTTP scenarios are the central tool: they are written once and executed against two storage back ends, in-memory repositories and PostgreSQL, through the production router. A browser test drives the real interface, and a load generator measures response times.

Table 6.1: Test levels

Level	Scope	Needs
Unit	Grade, attendance, timetable, news, export and access-control rules; validators	Nothing
Web	Sessions, CSRF, rendering, escaping, diary layout, snippets	Nothing
HTTP scenarios	Authentication, roles, five-point validation, notifications, attendance, CSV, news with script payloads, web login, cookie flags, rate limit	Nothing (in memory) or a migrated PostgreSQL database
Browser (E2E)	Main journeys of each role in a real browser	Running server
Performance	Latency percentiles under concurrent load	Running server, seeded database

All measurements in this chapter were taken on 19 October 2026 on one cloud virtual machine (2 vCPUs, 8 GB of memory, Linux), with Go 1.24.7 and PostgreSQL 16.15. The server, the database and the load generator shared this machine, which makes the latency figures conservative for the server and unsuitable as a statement about a dedicated deployment.

6.2 Automated test results

The suite contains 87 test functions, which run as 135 passing cases including subtests. With the race detector enabled no test fails and no data race is reported. The HTTP scenarios also pass on a real PostgreSQL 16 database (the scenario creates its own

administrator with a unique e-mail, so the database only has to be migrated and may already contain data). Statement coverage of the `internal` packages is 40.8 % when only in-memory tests run and 47.3 % when the PostgreSQL scenarios run as well, measured across packages. The figure is uneven: the service rules are covered well (the grade, export and access functions are covered at roughly 70 to 100 % of statements each), while the HTML page handlers are covered mainly by the browser test and show a low percentage in the Go tool. Coverage is therefore reported as a measure of what the automated Go tests reach, not as a measure of quality.

6.3 Verification of the five-point rule

The rule was exercised at the service, HTTP and database levels. Table 6.2 shows the API responses for representative inputs against the running server with PostgreSQL. All out-of-range and fractional values are rejected with 400 and the message “mark must be a whole number from 1 to 5”; the limits 1 and 5 are accepted. The check constraint was read back from the catalogue as `CHECK (marks = ANY (ARRAY[1,2,3,4,5]))`, so the same rule holds for any writer other than the application.

Table 6.2: Response of `POST /api/grades` to different mark values

Mark sent	HTTP status	Outcome
0	400	rejected
6	400	rejected
3.5	400	rejected
−1	400	rejected
1	201	recorded
5	201	recorded

The average and final mark are covered by unit tests with boundary values: 4.5 gives 5, 4.49 gives 4, an empty list gives no final mark, and the colour band function agrees with the rounding rule for 5, 4.5, 4.49, 3, 2, 1 and 0. The pupil’s marks page in Figure 5.5 shows the same numbers as the API report: for example four marks in mathematics (4, 5, 4, 4) give an average of 4.25 and a final mark of 4.

6.4 Migrations and backup

The five migrations were applied forwards, then backwards, then forwards again on an empty PostgreSQL 16 database without an error; the result contains 14 tables and 3 views.

The data conversion of migration 000003 is covered by its definition in Table 2.2; because its reverse is lossy by nature, the down script is documented and not used in normal operation.

A backup and restore round trip was performed on the large data set of Section 6.5. The dump of 256 KB was written, verified by listing and restored into a new empty database. The row counts of the original and the restored database were identical: 1 025 users, 12 269 marks, 20 240 attendance rows and 5 news items. A restore of a school-sized database therefore takes seconds here, far below the 30-minute objective, although a production database on slower storage will behave differently.

6.5 Load test

The data set for the load test contained 1 016 pupils, 12 269 marks and 20 240 attendance rows, created by the seeding tool and a SQL script. The load generator signs in as an administrator and then requests a list of endpoints from a given number of concurrent workers for ten seconds, recording latency percentiles and error rates. Table 6.3 shows the requirement level of 20 workers. Every endpoint is below the 200 ms target at the 95th percentile by a wide margin; the slowest is the pupil list at 53.8 ms. No request failed.

Table 6.3: Latency at 20 concurrent workers (10 s per endpoint, 1 016 pupils)

Endpoint	Requests/s	p50	p95	p99	Errors
/ (home page)	1 146	12.8 ms	54.2 ms	81.9 ms	0 %
`/api/news`	1 628	7.1 ms	41.5 ms	67.1 ms	0 %
`/api/classes`	2 250	3.6 ms	33.6 ms	56.6 ms	0 %
`/api/subjects`	1 912	4.5 ms	37.6 ms	69.2 ms	0 %
`/api/students`	990	16.4 ms	53.8 ms	84.3 ms	0 %
`/api/users`	1 334	10.9 ms	46.2 ms	72.3 ms	0 %

Figure 6.1 shows how the 95th percentile grows with the number of workers for five endpoints. Between one and ten workers it rises from about one millisecond to between 12 and 20 milliseconds; between 20 and 50 workers it grows slowly (for the home page from 54.2 to 58.3 ms) while throughput keeps increasing. The two-core machine is saturated well before 50 workers, so latency growth is queueing rather than a defect of an individual query. Even at 50 workers the highest p95 is 72.2 ms (`/api/students`).

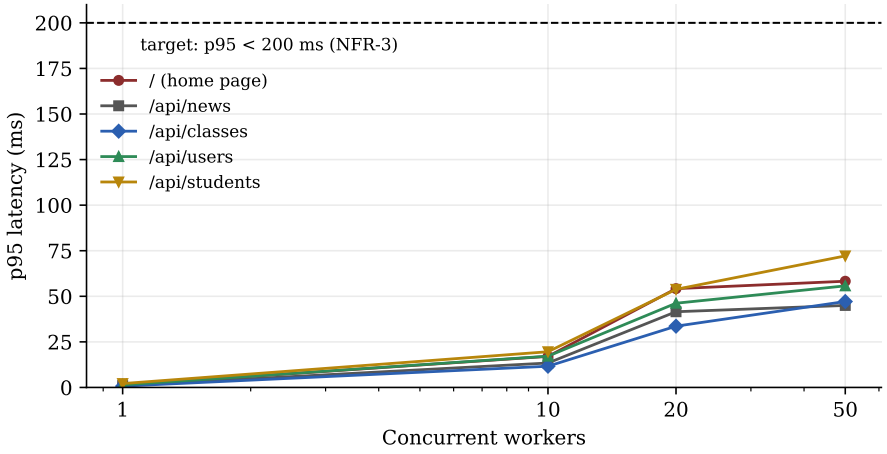


Figure 6.1: 95th-percentile latency against the number of concurrent workers

The result supports the stated requirement for this data size and configuration. It does not establish capacity for a larger school, for a database on separate hardware, for write-heavy periods such as the end of a term, or for repeated runs: each level was run once, so there is no estimate of run-to-run variation.

6.6 Browser and interface checks

The browser test (Playwright with Chromium) creates its own data through the administrator API and then walks through the main journeys: the guest sees the home page; an administrator publishes a news item whose title contains HTML tags, and the page shows the tags as text without interpreting them; a teacher gives a mark in the gradebook; a pupil opens the dashboard and the marks page; a pupil receives 403 when requesting an administrator page; a pupil downloads a CSV file; and the home page at phone width does not scroll horizontally. The test passed.

In addition, the main pages of the five kinds of user (guest, pupil, parent, teacher, administrator) were opened in a browser and the network and console were observed: 16 page loads produced no response with status 400 or higher and no console error. At a viewport of 390 px the document width equalled the viewport width, so there was no horizontal overflow. These are functional checks; they are not an accessibility audit, and contrast, keyboard operation and screen-reader behaviour were not formally evaluated.

6.7 Security checks

Table 6.4 lists the security behaviour that is covered by automated tests and by observation. The login limiter was also seen in operation during the load and browser runs: after more

than ten sign-ins within a minute from one address the server answered 429 until the window expired, which had to be worked around by restarting the server with a higher limit for the repeated tests.

Table 6.4: Security behaviour and how it was checked

Mechanism	Check
Closed registration	Public registration request returns 401; accounts exist only after administrator action
Role boundaries	Pupil cannot read another pupil; parent limited to linked children; pupil receives 403 on administrator pages
CSRF	State-changing web request without a matching `Origin` or `Referer` receives 403
Cookie flags	Session cookie is `HttpOnly` and `SameSite=Strict`
Script injection	News title and text with HTML are displayed as text (unit scenario and browser test)
CSV formula injection	Cells starting with `=`, `+`, `-` or `@` are prefixed in the export test
Response headers	Content-security policy, frame and content-type options present on responses
Login limit	Eleventh attempt within a minute returns 429 with `Retry-After`

6.8 Defects found during final verification

The final verification also found three problems, which were corrected. First, the news feed card showed only the first paragraph of an item, so a pinned announcement that began with a one-line greeting looked empty; the preview now collapses the whole text and a test covers it. Second, the continuous-integration definition in the repository still described the earlier test set and PostgreSQL 15; it was rewritten to use PostgreSQL 16 and to run the HTTP scenarios on a clean database, a backup and restore round trip, a short load test and the browser test. Third, the PostgreSQL scenario failed on a database that already contained an administrator, because it relied on the start-up bootstrap, which does nothing in that case. It now creates its own administrator directly in the repository; the corrected scenario passed on three databases, among them one with the demonstration data and one with the load-test data.

6.9 Assessment and limitations

The evaluation supports the intended workflow: marks on the five-point scale are validated at three levels and aggregated by a stated rule; the weekly diary and the home page work

for all roles; and the response-time requirement is met on a school-sized data set. The results are limited in the following ways.

- **No user study.** The usability test plan (five participants per role group, think-aloud tasks, task success and the System Usability Scale [11]) is prepared but was not carried out. Claims about ease of use, and the resemblance of the interface to the diary services that inspired it, rest on design reasoning and on the automated checks, not on user data.
- **Performance scope.** The load figures come from one shared two-core virtual machine, one run per level, with the load generator on the same host. They do not predict behaviour under larger data, separate hardware or write-heavy load.
- **Toolchain.** Tests were run with Go 1.24.7, while the module file and the container image target Go 1.21 and the pipeline pins that version. The compiler enforces the language level of the module file, and a search of the sources found none of the standard-library additions made after Go 1.21 that could be checked by name, but no build with a 1.21 toolchain was made here. The pipeline is the authority for compatibility with the older toolchain.
- **Coverage.** The page-handler layer is covered mostly by the browser test rather than by Go tests, and repository code is exercised only when the PostgreSQL scenarios run.
- **Deployment model.** The reminder job and the login limiter keep state in one process. Several instances would need shared scheduling and a shared limiter, and a reverse proxy must pass the client address correctly for the limiter to see individual clients.
- **Interface.** The interface is English only, news is plain text without formatting or attachments, and there is no accessibility audit.
- **Data conversion.** The conversion of legacy 0–100 scores to marks is lossy and irreversible in practice. It is appropriate for a demonstration database; a school with real historical scores would have to agree the bands with its teachers first.

7 Conclusion and future work

7.1 Application outcomes

The project produced a working web application for school management with a five-point marking scheme, a weekly electronic diary and a public home page with school news. Administrators manage accounts, classes, subjects, the timetable, news and announcements. Teachers record attendance and give marks with a single press in a gradebook. Pupils and parents read a weekly diary in which each mark stands next to its lesson, a marks page with averages and final marks, attendance, notifications and CSV exports. A visitor who is not signed in sees the latest news.

The central technical contribution is a set of rules that are explicit and tested rather than implicit in the screens. Access is decided by one component from relationships stored in the database. A mark is a whole number from 1 to 5, enforced by the service and by a database constraint. The subject mark is the average rounded half up, and the attendance percentage has a stated formula with a one-time low-attendance warning. The migration from the earlier 0–100 scores has defined, documented behaviour in both directions.

The architecture keeps handlers, rules and persistence in separate layers, and services depend on interfaces. This allowed the same HTTP scenarios to run on in-memory repositories and on PostgreSQL through the production router, which gave the tests the same routes, middleware and handlers as the deployed system.

7.2 Testing results

The suite of 87 test functions (135 passing cases) passes with the race detector, and the HTTP scenarios pass on PostgreSQL 16. The migrations apply forwards, backwards and forwards again, and a backup restores into an identical database. With 1 016 pupils, 12 269 marks and 20 240 attendance rows and 20 concurrent workers, every measured endpoint answered with a 95th-percentile latency of at most 54.2 ms and without errors, against a requirement of 200 ms. The browser test and a review of the main pages of every role found no functional errors, no horizontal overflow at phone width and correct escaping of hostile text.

These results answer the evaluation questions within their stated conditions. The rules hold under test, the performance requirement is met for a school-sized data set on the

measured configuration, and the limits listed in Section 6.9 define what remains unproven, above all the absence of a user study.

7.3 Possible extensions

The most valuable next step is the usability study that was planned but not run: five participants per role group, the defined tasks and the System Usability Scale, which would turn design intentions about the diary into evidence. A second step is to make the interface multilingual, starting with Hungarian and Russian, which is mechanical because all text is in templates.

Several functional extensions follow from the design. Homework and lesson topics could be added to the diary, since the week layout already has the place for them. News could support formatting, pictures and visibility by role or class. Messages between teachers and parents would need moderation rules. Fee tracking, which the original plan listed as optional, would reuse the same access component. On the technical side, a deployment with several processes would require a shared scheduler for reminders and a shared rate limiter, and an accessibility audit would show how well the interface serves users with different needs.

7.4 Final conclusion

The system combines persistent storage, explicit access and grading rules, and a diary-style interface in a usable school application. Its engineering decisions are supported by automated tests, a load measurement against a stated requirement, and a recorded list of limitations. The main outcome is a working application whose rules can be read in the code, checked by the tests and explained to the people who depend on them.

Bibliography

[1] Zviagintsev, M. School management system: source code, migrations, tests and documentation. https://github.com/ringogallagher/school-management. Accessed 19 October 2026.

[2] PostgreSQL Global Development Group. PostgreSQL 16 documentation: Constraints. https://www.postgresql.org/docs/16/ddl-constraints.html. Accessed 19 October 2026.

[3] PostgreSQL Global Development Group. PostgreSQL 16 documentation: pg_dump. https://www.postgresql.org/docs/16/app-pgdump.html. Accessed 19 October 2026.

[4] Jones, M., Bradley, J. and Sakimura, N. JSON Web Token (JWT). RFC 7519, Internet Engineering Task Force, May 2015. https://www.rfc-editor.org/rfc/rfc7519. Accessed 19 October 2026.

[5] Provos, N. and Mazières, D. A future-adaptable password scheme. *Proceedings of the USENIX Annual Technical Conference*, 1999.

[6] OWASP Foundation. Cross-Site Request Forgery Prevention Cheat Sheet. https://cheatsheetseries.owasp.org/cheatsheets/Cross-Site_Request_Forgery_Prevention_Cheat_Sheet.html. Accessed 19 October 2026.

[7] OWASP Foundation. CSV Injection. https://community.owasp.org/attacks/CSV_Injection. Accessed 19 October 2026.

[8] MDN contributors. Content Security Policy (CSP). https://developer.mozilla.org/en-US/docs/Web/HTTP/CSP. Accessed 19 October 2026.

[9] htmx contributors. htmx documentation. https://htmx.org/docs/. Accessed 19 October 2026.

[10] golang-migrate contributors. migrate: database migrations. https://github.com/golang-migrate/migrate. Accessed 19 October 2026.

[11] Brooke, J. SUS: a "quick and dirty" usability scale. In Jordan, P. W. et al. (eds.), *Usability Evaluation in Industry*. Taylor & Francis, London, 1996, pp. 189–194.

Declaration on the Use of Artificial Intelligence Tools

I hereby declare that during the preparation of my thesis I have used the following artificial intelligence-based tools.

1. Use of AI tools

☐ I used AI tools as follows:

- **Claude (Anthropic):** assistance with writing and understanding the project code, with writing and running tests and verification runs, with the conversion of the application to the five-point marking scheme, the diary-style interface and the news home page, and with drafting and revising the thesis text. Claude was used through Claude Code and the Claude application; several Claude models were used over the course of the work.
- No other artificial intelligence-based tools were used.

2. Declaration of independent work

I declare that I take full responsibility for the professional content of my thesis. I acknowledge that the use of artificial intelligence-based tools does not replace independent work; therefore, I have verified all statements, results, interpretations, conclusions and references included in the thesis.

I further declare that:

- the final content of the thesis was compiled by me,
- any AI-generated or modified content has been reviewed, verified and corrected where necessary,
- the original results, interpretations and conclusions presented in the thesis are my responsibility,
- the use of AI tools has been truthfully and fully disclosed in this declaration.